



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	Cindy Aulia
NPM	2428240134
Kelas	SI5C
Topik Tugas 1	Properti
Entitas utama	Properti
Tautan repositori	https://github.com/cindyauliaMDP/si5c_kuis1_CindyAulia
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	8 oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis2_SI5C_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Data topik

Topik Tugas 1	Properti
Entitas utama (tunggal)	Properti
Alamat dasar	/properties
Field data	id, judul, tipe, kota, luasM2, harga
Field wajib saat POST	judul, tipe, kota, luasM2, harga
Query string untuk filter	kota
Data awal	3 data properti
Modul lain (jika ada)	

Daftar endpoint

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/properties	Tidak	200	Menampilkan seluruh data property dan dapat melakukan filter berdasarkan kota
2	GET	/ properties /:id	Tidak	200	Menampilkan satu property berdasarkan ID
3	POST	/properties	Ya	201	Menambah data properti baru
4	PUT	/ properties /:id	Ya	200	Mengubah data property berdasarkan ID
5	DELETE	/ properties /:id	Ya	204	Menghapus data property berdasarkan ID

Bab 1 Instalasi

1.1 Instalasi project

```

v24.21.0
PS C:\PAW2\si5c_kuis1_CindyAulia> npm -v
11.19.0
PS C:\PAW2\si5c_kuis1_CindyAulia> npm init -y
Wrote to C:\PAW2\si5c_kuis1_CindyAulia\package.json:

{
  "name": "si5c_kuis1_cindyaulia",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/cindyaulia0P/si5c_kuis1_CindyAulia.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "bugs": {
    "url": "https://github.com/cindyaulia0P/si5c_kuis1_CindyAulia/issues"
  },
  "homepage": "https://github.com/cindyaulia0P/si5c_kuis1_CindyAulia#readme"
}

PS C:\PAW2\si5c_kuis1_CindyAulia> npm install express cors dotenv
added 71 packages, and audited 72 packages in 6s
30 packages are looking for funding
run 'npm fund' for details
found 0 vulnerabilities
PS C:\PAW2\si5c_kuis1_CindyAulia>

```

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

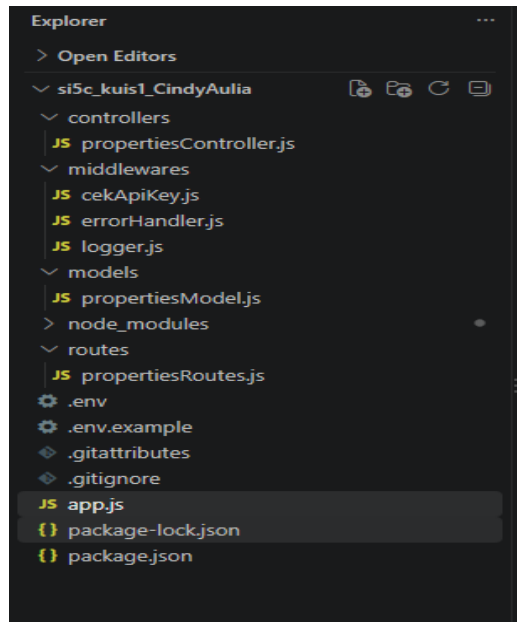
Pada tahap instalasi, project menggunakan Node.js versi 24.21.0 dan npm versi 11.19.0. Project dibuat menggunakan perintah `npm init -y`, kemudian menginstal beberapa package yang dibutuhkan, yaitu `express`, `cors`, dan `dotenv`.

Express digunakan sebagai framework untuk membangun server dan RESTful API. CORS digunakan untuk mengatur akses request dari origin yang berbeda. `dotenv` digunakan untuk membaca variabel lingkungan yang disimpan pada file `.env`, salah satunya adalah API key.

Versi Node.js yang digunakan sudah memenuhi kebutuhan Express 5 karena Express 5 membutuhkan Node.js versi 18 atau lebih tinggi.

Bab 2 Struktur Folder dan Ringkasan Kode

2.1 Struktur folder



Gambar 2. Struktur folder project

Penjelasan:

Project RESTful API ini menggunakan arsitektur backend terstruktur yang terdiri dari beberapa folder, yaitu routes, controllers, models, dan middlewares.

1. routes digunakan untuk menentukan endpoint dan method HTTP yang digunakan.
2. controllers digunakan untuk menangani request dan response serta logika proses API.
3. models digunakan untuk mengelola data property.
4. middlewares digunakan untuk menjalankan proses tambahan sebelum atau sesudah request diproses, seperti logger, pengecekan API key, dan error handler.

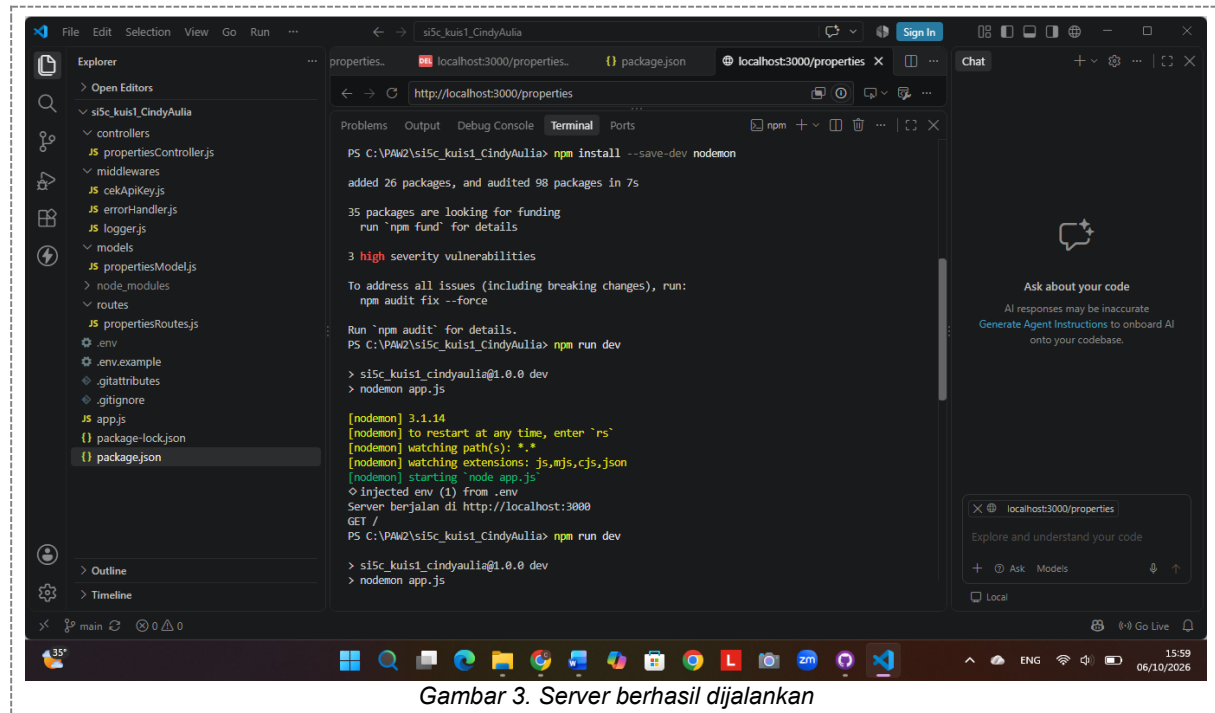
2.2 Ringkasan tugas setiap berkas

Berkas	Penjelasan Anda
models/propertiesModel.js	Berkas ini digunakan untuk mengelola data property. Di dalamnya terdapat fungsi getAll(), getByld(), create(), update(), dan remove() untuk mengambil, menambah, mengubah, dan menghapus data property.

Berkas	Penjelasan Anda
controllers/propertiesController.js	Berkas ini menangani request dan response dari endpoint property. Controller memiliki fungsi getAll(), getByld(), create(), update(), dan remove(). Validasi data POST juga dilakukan pada bagian controller.
routes/propertiesRoutes.js	Berkas ini mendefinisikan endpoint RESTful API untuk property, yaitu GET, POST, PUT, dan DELETE. Route juga menghubungkan request dengan fungsi pada propertiesController.
middlewares/logger.js	Middleware ini digunakan untuk mencatat method HTTP dan alamat URL yang diakses. Informasi tersebut ditampilkan melalui console.log().
middlewares/cekApiKey.js	Middleware ini digunakan untuk memeriksa header x-api-key. API key yang dikirim akan dibandingkan dengan nilai API_KEY yang disimpan pada environment.
middlewares/errorHandler.js	Middleware ini digunakan untuk menangani error secara terpusat. Salah satu penerapannya adalah menangani JSON yang tidak valid sehingga API memberikan response JSON dengan status 400 dan pesan "Format JSON tidak valid".
app.js	Berkas app.js merupakan file utama aplikasi. File ini memuat konfigurasi dotenv, Express, CORS, express.json(), middleware, routes, dan menjalankan server pada port 3000.

Bab 3 Menjalankan Server

3.1 Menjalankan server



Gambar 3. Server berhasil dijalankan

Penjelasan:

Server dijalankan menggunakan file app.js dan menggunakan port 3000. Nomor port tersebut berasal dari konstanta PORT yang ditentukan pada file app.js. Ketika server berhasil dijalankan, terminal menampilkan pesan:

"Server berjalan di http://localhost:3000"

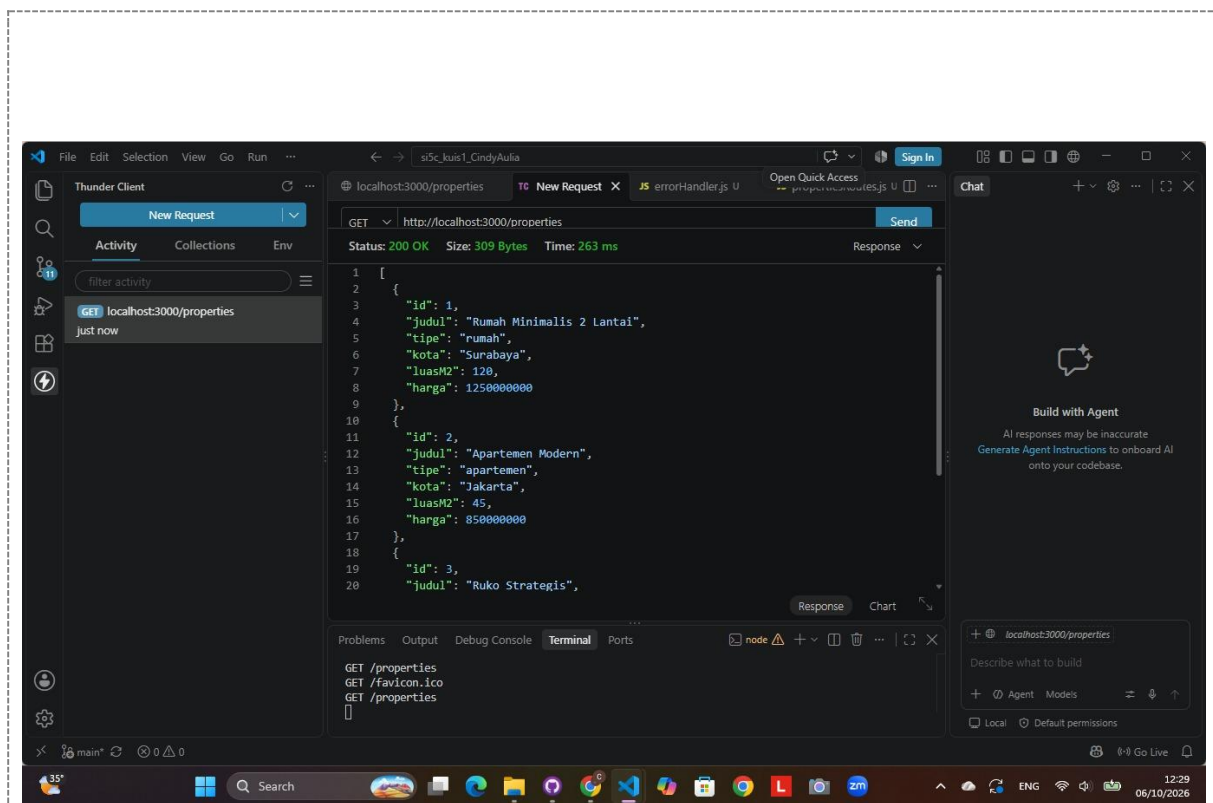
Server harus tetap berjalan selama proses pengujian endpoint dilakukan. Apabila port 3000 sedang digunakan oleh aplikasi lain, server tidak dapat menggunakan port tersebut sehingga akan terjadi error. Solusinya adalah menghentikan proses yang menggunakan port tersebut atau menggunakan nomor port yang berbeda.

Bab 4 Pengujian Setiap Endpoint

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /properties)

Permintaan (<i>request</i>)	GET /properties
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; dan response berupa array JSON yang berisi data property.
Status hasil uji	200 OK
Lulus?	[✓] Ya [] Tidak



Gambar 4. Hasil GET semua data

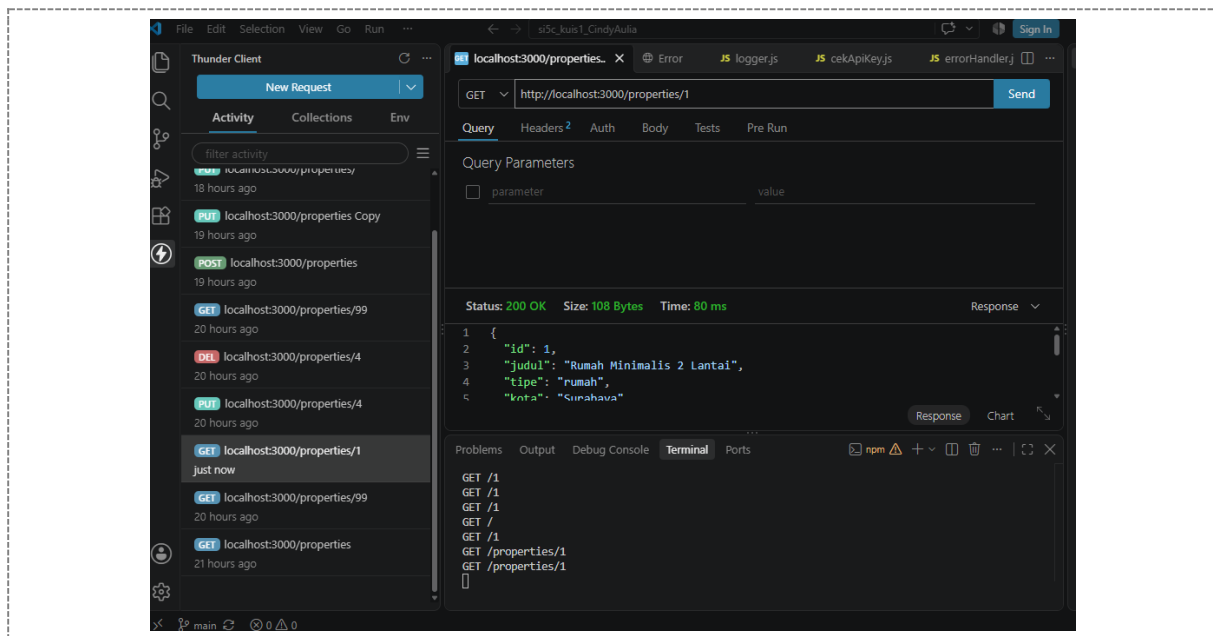
Penjelasan:

Endpoint GET /properties digunakan untuk mengambil seluruh data property. Request pertama kali diproses oleh route /properties, kemudian route memanggil fungsi getAll() pada propertiesController.

Controller mengambil query kota melalui req.query. Setelah itu controller memanggil propertiesModel.getAll(kota) untuk mengambil data dari model. Jika parameter kota tidak diberikan, seluruh data property akan ditampilkan.

T02 Menampilkan satu data berdasarkan ID

Permintaan (request)	GET / properties /1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	200 OK
Lulus?	[✓] Ya [] Tidak



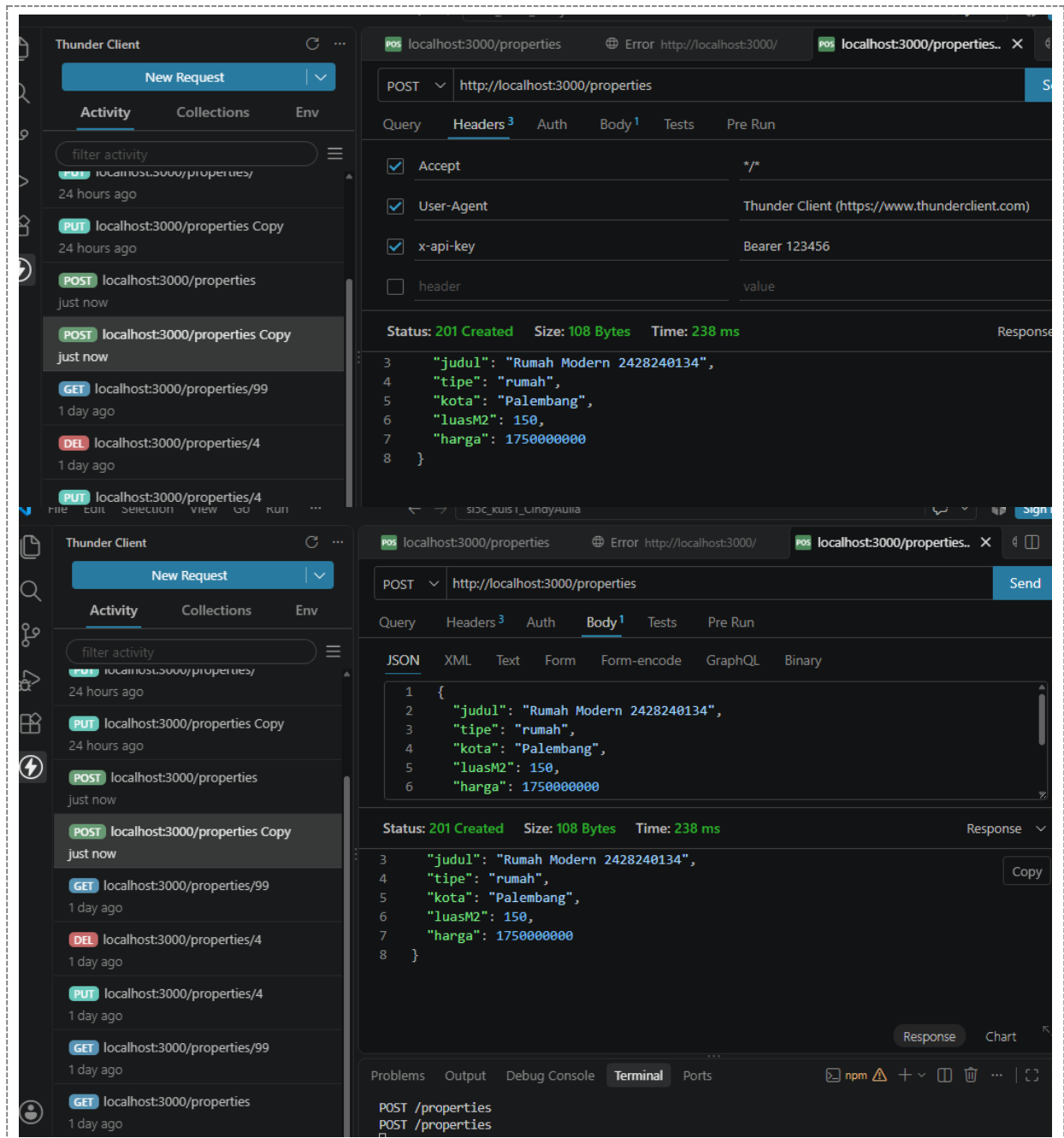
Gambar 5. Hasil GET berdasarkan ID

Penjelasan:

Endpoint GET /properties/:id digunakan untuk mengambil satu data property berdasarkan ID. ID diambil menggunakan req.params.id. Karena nilai parameter dari URL berupa string, ID kemudian diubah menjadi angka menggunakan parseInt() agar dapat dibandingkan dengan ID pada data property yang bertipe angka. Selanjutnya controller memanggil propertiesModel.getById(id) untuk mencari property berdasarkan ID tersebut.

T03 Menambah data baru (POST)

Permintaan (<i>request</i>)	POST / properties
Header	x-api-key: Bearer 123456 Content-Type: application/json
Body	{ "judul": "Rumah Modern 2428240134", "tipe": "rumah", "kota": "Palembang", "luasM2": 150, "harga": 1750000000 }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created
Lulus?	☒ Ya ☐ Tidak



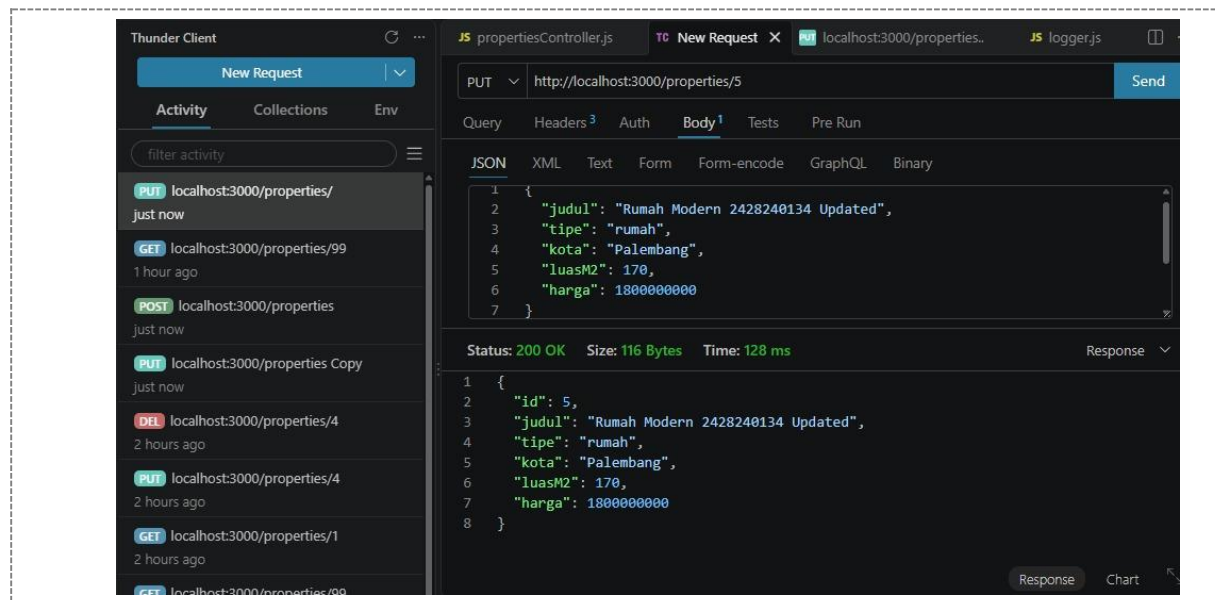
Gambar 6. Hasil POST data baru

Penjelasan:

Pada endpoint POST, request terlebih dahulu melewati middleware cekApiKey. Middleware tersebut memeriksa header x-api-key untuk memastikan API key yang dikirim sesuai dengan API key pada environment. Express.json() digunakan untuk membaca body request dalam format JSON. Setelah API key berhasil diperiksa, request diteruskan ke fungsi create() pada controller. Controller melakukan validasi terhadap field wajib, kemudian memanggil propertiesModel.create() untuk membuat data baru. Status 201 Created digunakan karena request berhasil membuat resource atau data property baru.

T04 Mengubah data (PUT)

Permintaan (request)	PUT / properties /5
Header	x-api-key: Bearer 123456 Content-Type: application/json
Body	{ "judul": "Rumah Modern 2428240134 Updated", "luasM2": 170, "harga": 1800000000 }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK
Lulus?	[✓] Ya [] Tidak



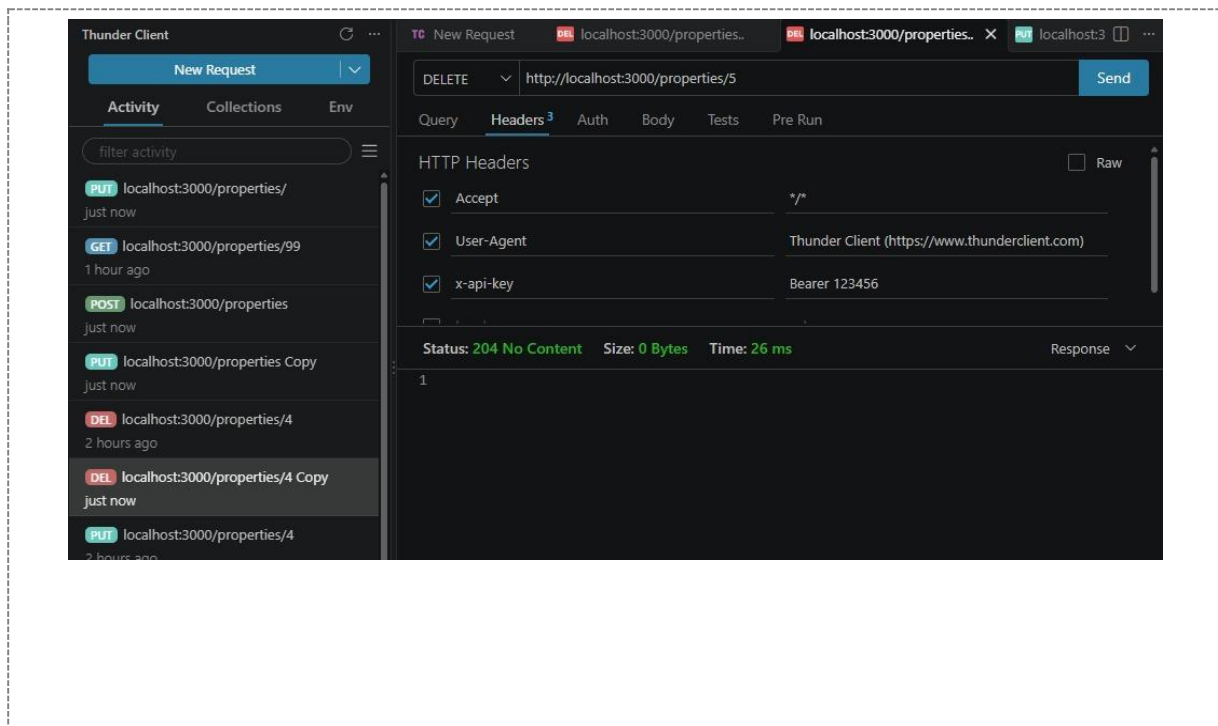
Gambar 7. Hasil PUT mengubah data

Penjelasan:

Endpoint PUT digunakan untuk mengubah data property berdasarkan ID yang dibuat pada pengujian T03. Request terlebih dahulu melewati middleware cekApiKey. Setelah API key valid, controller mengambil ID dari req.params.id dan data perubahan dari req.body. Controller kemudian memanggil fungsi update() pada model. Pada pengujian ini, data judul, luasM2, dan harga diubah, sedangkan data lainnya tetap.

T05 Menghapus data (DELETE)

Permintaan (request)	DELETE /properties/5
Header	x-api-key: Bearer 123456
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	204 No Content
Lulus?	[✓] Ya [] Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

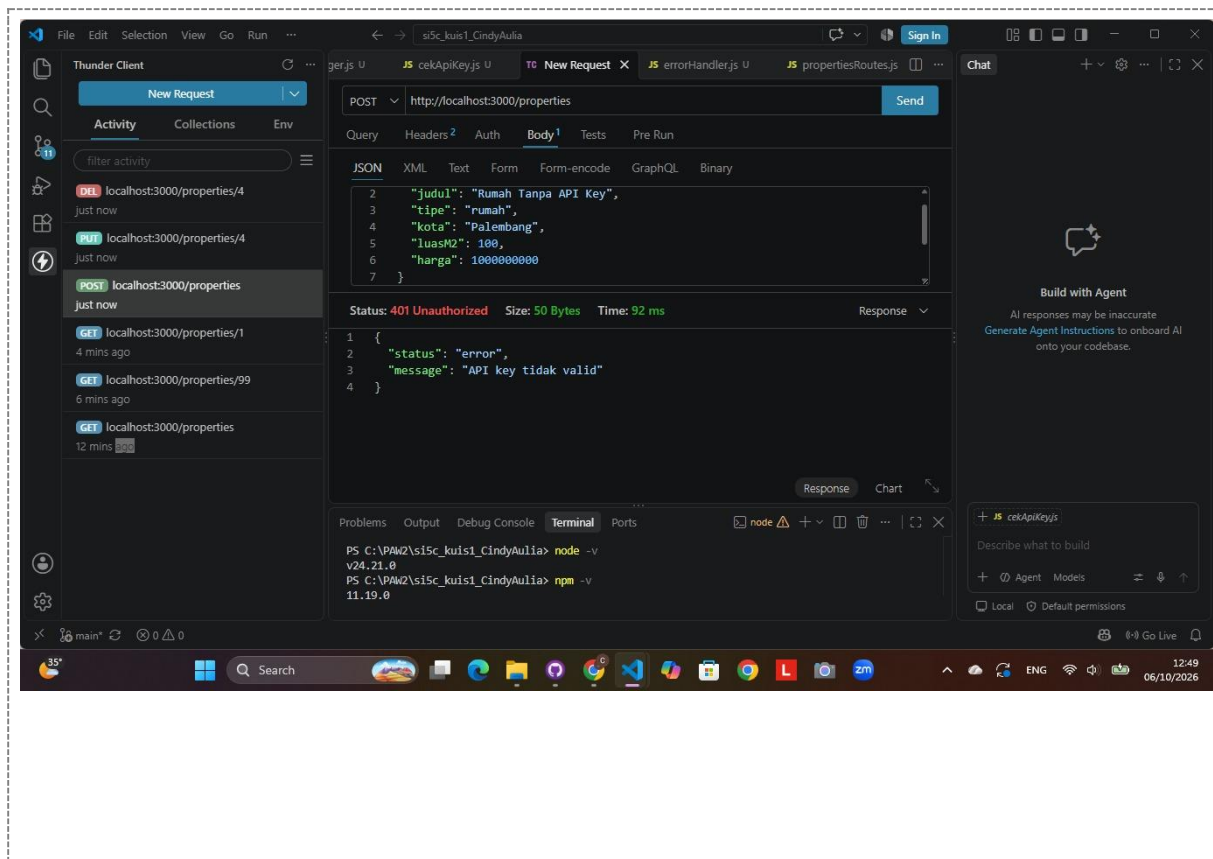
Endpoint DELETE digunakan untuk menghapus data property berdasarkan ID. Request melewati middleware cekApiKey, kemudian controller memanggil fungsi remove() pada model. Jika data berhasil dihapus, server memberikan status 204 No Content. Status 204 menunjukkan bahwa proses berhasil dilakukan tetapi server tidak mengirimkan isi response.

Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Permintaan (request)	POST /[sumber-daya] (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ "judul": "Rumah Tanpa API Key", "tipe": "rumah", "kota": "Palembang", "luasM2": 100, "harga": 1000000000 }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized
Lulus?	[✓] Ya [] Tidak



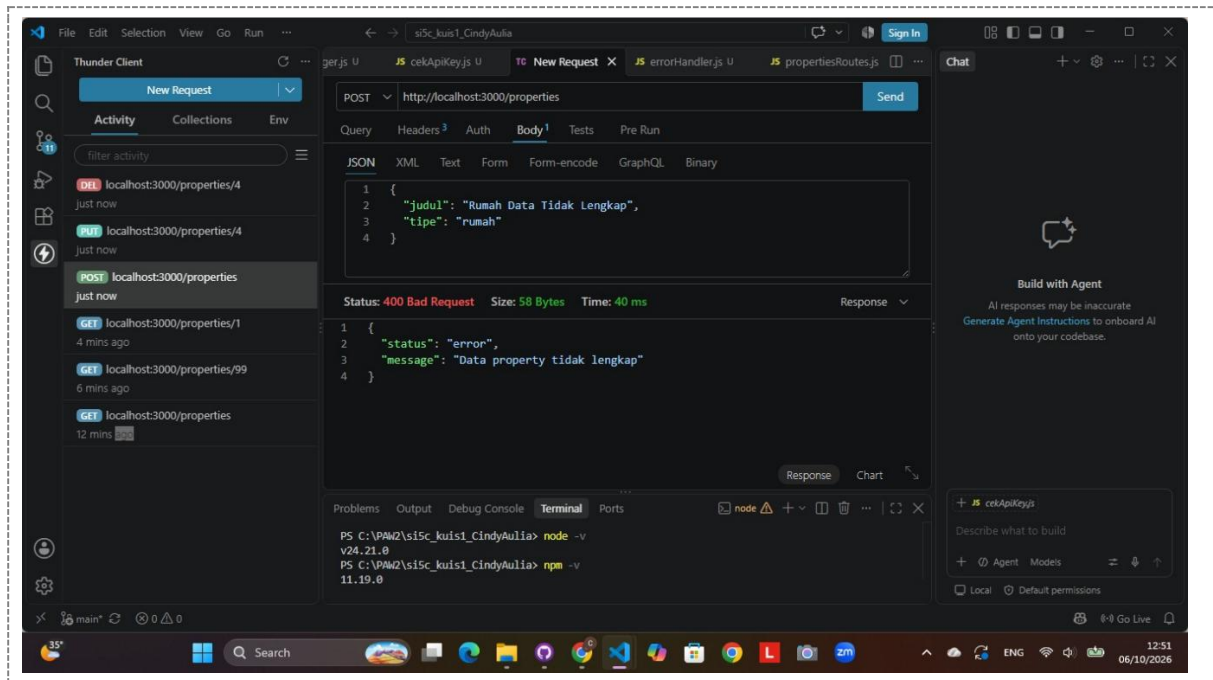
Gambar 9. POST tanpa API key ditolak

Penjelasan:

Request ditolak oleh middleware cekApiKey karena tidak terdapat header x-api-key yang valid. Middleware mengembalikan response dengan status 401 Unauthorized, sehingga request tidak diteruskan ke controller. Dengan demikian, proses penambahan data tidak dilakukan.

N02 Menambah data dengan body tidak lengkap

Permintaan (request)	POST /properties
Header	x-api-key: Barrier 123456 Content-Type: application/json
Body	<pre>{ "judul": "Rumah Data Tidak Lengkap", "tipe": "rumah"}</pre>
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request
Lulus?	[✓] Ya [] Tidak



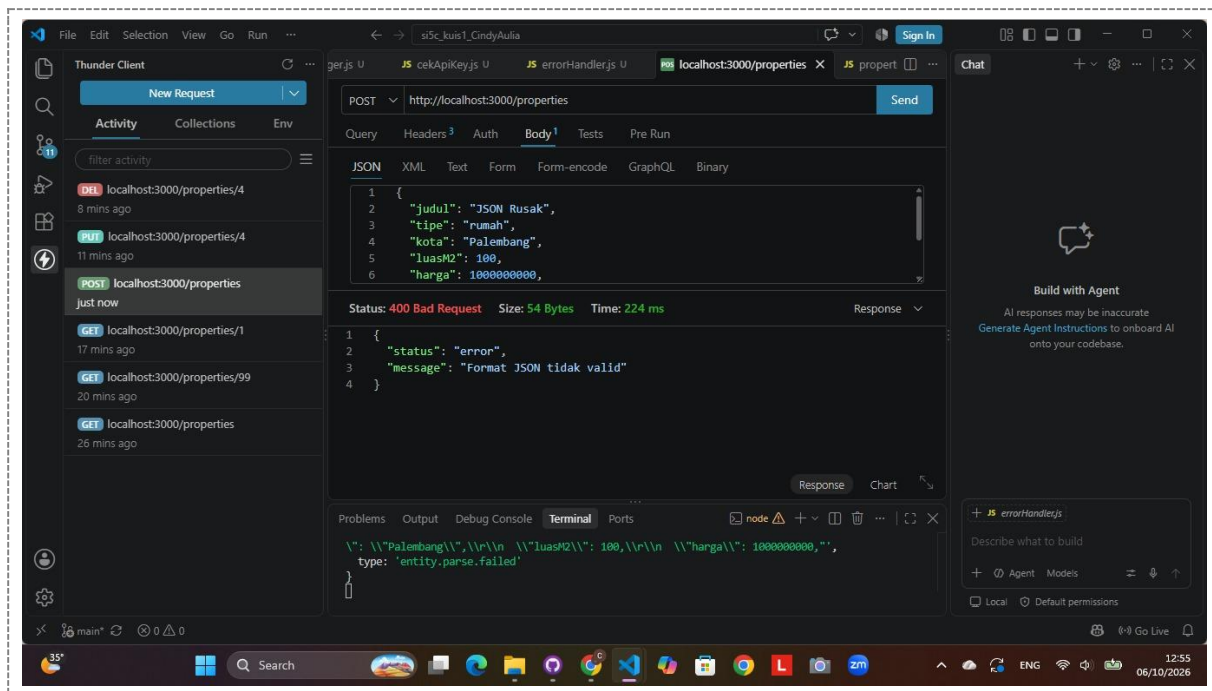
Gambar 10. POST dengan body tidak lengkap

Penjelasan:

Validasi body dilakukan pada fungsi create() di propertiesController.js. Field yang wajib dikirim adalah judul, tipe, kota, luasM2, dan harga. Jika salah satu field wajib tidak tersedia, controller mengembalikan response dengan status 400 Bad Request. Status 400 digunakan karena kesalahan berasal dari data request yang dikirim oleh client.

N03 Mengirim JSON yang rusak

Permintaan (request)	POST /[sumber-harga]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "judul": "JSON Rusak", "tipe": "rumah", "kota": "Palembang", "luasM2": 100, "harga": 1000000000 }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request
Lulus?	[✓] Ya [] Tidak



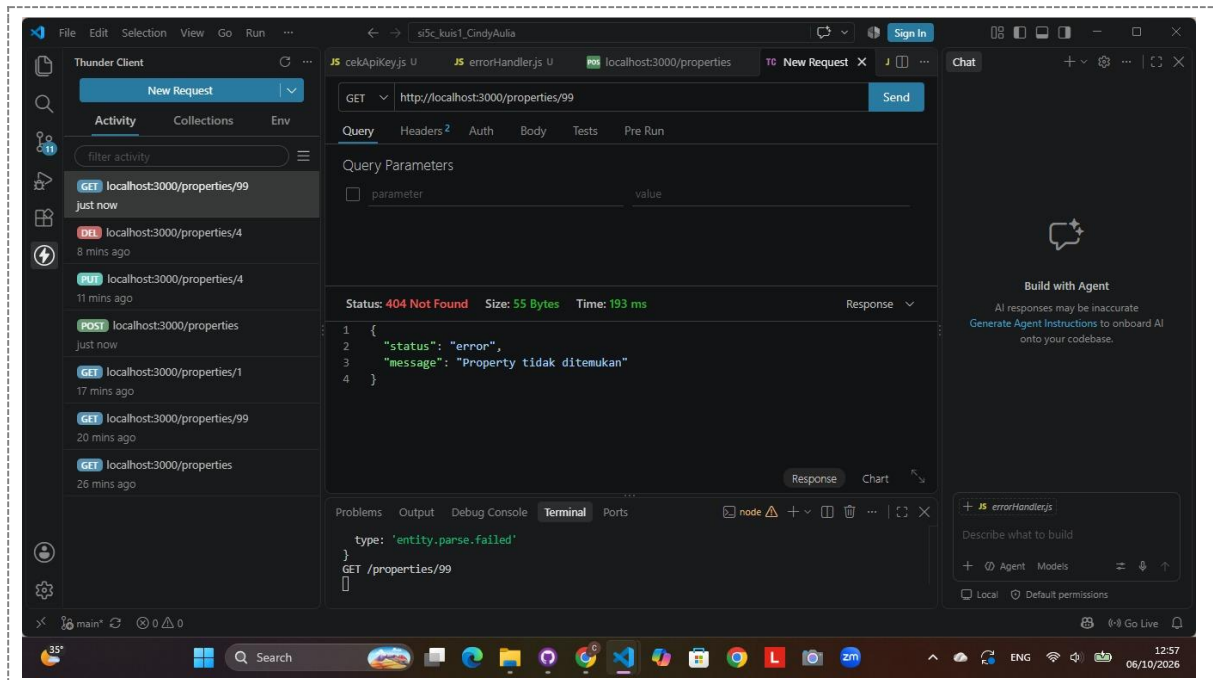
Gambar 11. POST dengan JSON rusak

Penjelasan:

JSON yang rusak dideteksi ketika middleware `express.json()` mencoba membaca body request. Karena JSON tidak dapat diproses, error diteruskan ke `errorHandler`. Middleware `errorHandler` kemudian mengenali error tersebut dan mengubahnya menjadi response JSON dengan status 400 dan pesan "Format JSON tidak valid". Dengan demikian, API tidak menampilkan halaman error HTML, tetapi memberikan response JSON yang lebih sesuai untuk RESTful API.

N04 Mengambil ID yang tidak ada

Permintaan (request)	GET /properties/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found
Lulus?	[✓] Ya [] Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Request GET /properties/99 diproses oleh route berdasarkan ID dan diteruskan ke fungsi getByld() pada controller. Controller mengambil ID 99 dari req.params.id, kemudian memanggil propertiesModel.getByld(id). Karena ID tersebut tidak ditemukan pada data property, controller mengembalikan response dengan status 404 Not Found dan pesan "Property tidak ditemukan".

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200	✓
T02	Menampilkan satu data berdasarkan ID	200	200	✓
T03	Menambah data baru	201	201	✓
T04	Mengubah data	200	200	✓
T05	Menghapus data	204	204	✓
N01	Menambah data tanpa API key	401	401	✓
N02	Menambah data dengan body tidak lengkap	400	400	✓
N03	Mengirim JSON yang rusak	400	400	✓
N04	Mengambil ID yang tidak ada	404	404	✓

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/cindyauliaMDP/si5c_kuis1_CindyAulia
Pesan commit terakhir	Laporan pdf (terlambat commit laporan, Maaf ya Pak 🙏)
Folder routes, controllers, models, middlewares ada	☒ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☒ Ya ☐ Tidak
.env.example ada	☒ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Route digunakan untuk menentukan alamat atau jalur akses pada API beserta metode yang digunakan, seperti GET atau POST. Misalnya, pada file `propertiesRoutes.js` terdapat `router.get('/properties', propertiesController.getAll)` yang berfungsi untuk mengarahkan permintaan ke data properti.

Controller berperan sebagai penghubung yang memproses permintaan dari pengguna dan memberikan respon. Controller juga mengatur alur jalannya proses di dalam API. Contohnya, fungsi `getAll()` di dalam `propertiesController.js` bertugas untuk menangani permintaan untuk menampilkan data.

Model bertugas untuk mengelola data secara langsung, seperti mengambil atau menyimpan data properti ke database. Contohnya, pada `propertiesModel.getAll(kota)` digunakan untuk mengambil data properti berdasarkan kota yang diminta.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Error handler terpusat berfungsi agar format pesan error lebih konsisten tanpa harus ditulis di setiap route. Contohnya pada pengujian N03 ketika JSON yang dikirim rusak, maka akan dikembalikan respon JSON dengan status 400 dan pesan "Format JSON tidak valid".

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	✓
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	✓
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	✓
4	Identitas saya (nama atau NIM) muncul pada data uji POST	✓
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	✓
6	Tabel rekapitulasi sesuai dengan tangkapan layar	✓
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	✓
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	✓

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100